OFFLINE, PRIVACY-PRESERVING HINDI VOICE ASSISTANT ON RASPBERRY PI

A REPORT

Submitted by

Pushkar Chaturvedi (231B240)

Rishabh Jain (231B264)

Under the Guidance of: Mr. Ajay Kumar

Submitted to

Bharat AI-SoC Student Challenge

February 2026

JAYPEE UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Department of Computer Science & Engineering

Abstract

This project presents the design and implementation of a fully offline Hindi Voice Assistant optimized for ARM-based edge devices. The primary objective was to develop a privacy-preserving, low-latency voice interaction system capable of running entirely on resource-constrained hardware without reliance on cloud services.

The system integrates streaming-based Automatic Speech Recognition (ASR) using the Vosk engine, a lightweight rule-based Natural Language Processing (NLP) module for intent detection, and an offline Text-to-Speech (TTS) engine for response generation. A modular software architecture was implemented to ensure scalability, maintainability, and efficient resource utilization.

The assistant was deployed and tested on an ARM-based single-board computer (RPi-4B), demonstrating real-time performance with optimized CPU and memory usage. By eliminating cloud dependencies, the system ensures data privacy, reduced network overhead, and improved reliability in low-connectivity environments.

This project highlights the practical application of Edge AI and demonstrates how intelligent systems can be effectively implemented on AI-SoC platforms.

Table of Contents

1. Problem Statement

1.1. Background & Motivation

Voice assistants today largely depend on cloud-based processing for speech recognition and natural language understanding. While effective, such systems introduce several limitations:

- Dependence on constant internet connectivity
- Increased latency due to cloud communication
- Privacy concerns related to voice data transmission
- Limited usability in low-connectivity environments

In countries like India, where multilingual support and rural deployment are essential, offline and edge-based AI solutions become critically important.

1.2. Objectives

The primary objectives of this project were:

- To develop a fully offline Hindi voice assistant
- To ensure real-time processing on ARM-based hardware
- To design a modular and scalable software architecture
- To optimize memory and CPU utilization for edge deployment
- To eliminate cloud dependency and enhance privacy

1.3. Scope of the Project

The project focuses on:

- Real-time speech recognition (Hindi)
- Intent detection using lightweight NLP techniques
- Execution of predefined actions (e.g., timers, responses)
- Offline text-to-speech generation
- Deployment and optimization on an ARM-based SoC

The system is designed for extensibility and future integration with smart devices and advanced AI models.

2. System Overview

2.1. Overall Architecture

The system follows a sequential processing pipeline:

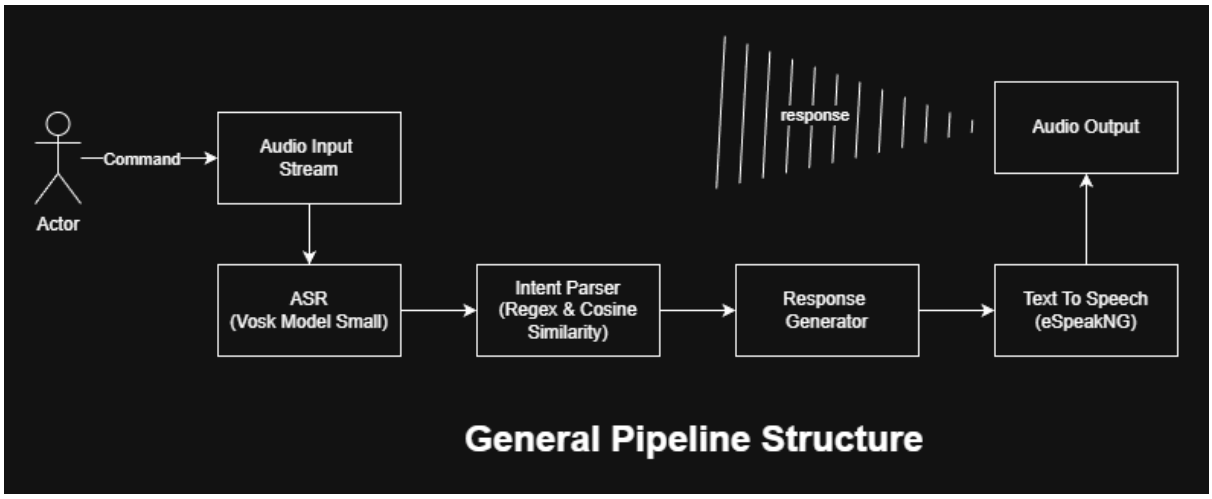

Figure 1: *General Pipeline Structure*

The architecture is modular, ensuring separation of concerns between audio processing, speech recognition, language understanding, and response generation.

2.2. Key Components

The system consists of the following major components:

- Audio Input Module
- Automatic Speech Recognition (ASR) Engine
- NLP-Based Intent Parser
- State Management System
- Action Execution Layer
- Text-to-Speech Engine

Each module operates independently but integrates through a unified control flow.

2.3. Technology Stack

- **Programming Language:** Python
- **ASR Engine:** Vosk (Offline Hindi Model)
- **NLP:** Rule-based intent detection
- **TTS:** Offline speech synthesis engine
- **Hardware Platform:** ARM-based Single Board Computer (RPI4)
- **Version Control:** Git & GitHub

Vosk was selected due to its lightweight architecture, ARM compatibility, and offline capability.

3. Implementation

3.1. Hardware Setup & System Configuration

1. Raspberry Pi 4 Model B

Figure 2: *Raspberry Pi 4 Model B*

The *Raspberry Pi 4B* is a low-cost, credit-card-sized single board computer (SBC) developed by the Raspberry Pi Foundation. It integrates a processor, memory, networking, and input/output interfaces on a single printed circuit board (PCB), enabling it to function as a complete microcomputer. It is widely used in embedded systems, IoT & edge computing applications.

Key Specifications:

Parameter	Specification
Developer	Raspberry Pi Foundation
SoC	Broadcom BCM2711
CPU	Quad-core ARM Cortex-A72 (64-bit)
Clock Speed	1.5 GHz
RAM Options	8GB LPDDR4
GPIO Header	40-pin

Parameter	Specification
Wireless	Dual-band Wi-Fi, Bluetooth 5.0
Ethernet	Gigabit Ethernet
USB Ports	2 × USB 3.0, 2 × USB 2.0
Audio Interface	PWM Audio + I2S Support
Operating Voltage	5V (via USB-C)
OS Support	Raspberry Pi OS (Linux-based)

2. INMP441 MEMS High Precision

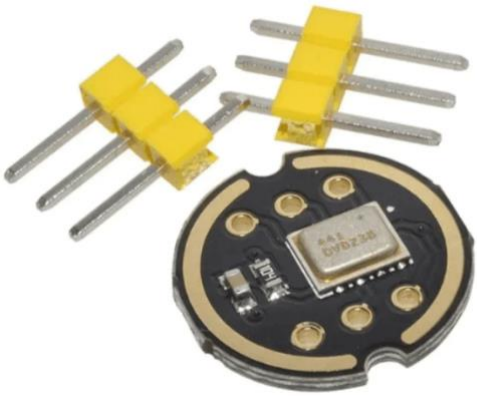

Figure 3: *INMP441 MEMS High Precision*

- The INMP441 is a high-performance, low-power, digital-output, omnidirectional MEMS microphone with a bottom port.

- The complete INMP441 solution consists of a MEMS sensor, signal conditioning, analog-to-digital converter, anti-aliasing filter, power management, and an industry-standard 24-bit I2S interface.

- The I2S interface allows the INMP441 to be directly connected to digital processors such as DSPs and microcontrollers without the need for an external audio codec.

- The INMP441 has a high signal-to-noise ratio and is an excellent choice for near-field applications. It features a flat wideband frequency response, resulting in high-definition natural sound.

Key Specifications:

Feature	Technical Specification
Digital Output Interface	I2S (Inter-IC Sound) with 24-bit resolution
Signal-to-Noise Ratio (SNR)	61 dBA
Sensitivity	−26 dBFS
Frequency Response	60 Hz – 15 kHz
Current Consumption	1.4 mA
Power Supply Rejection (PSR)	−75 dBFS

3. Hardware Connection Between Raspberry Pi 4B and INMP441

a. GPIO Pin Connections

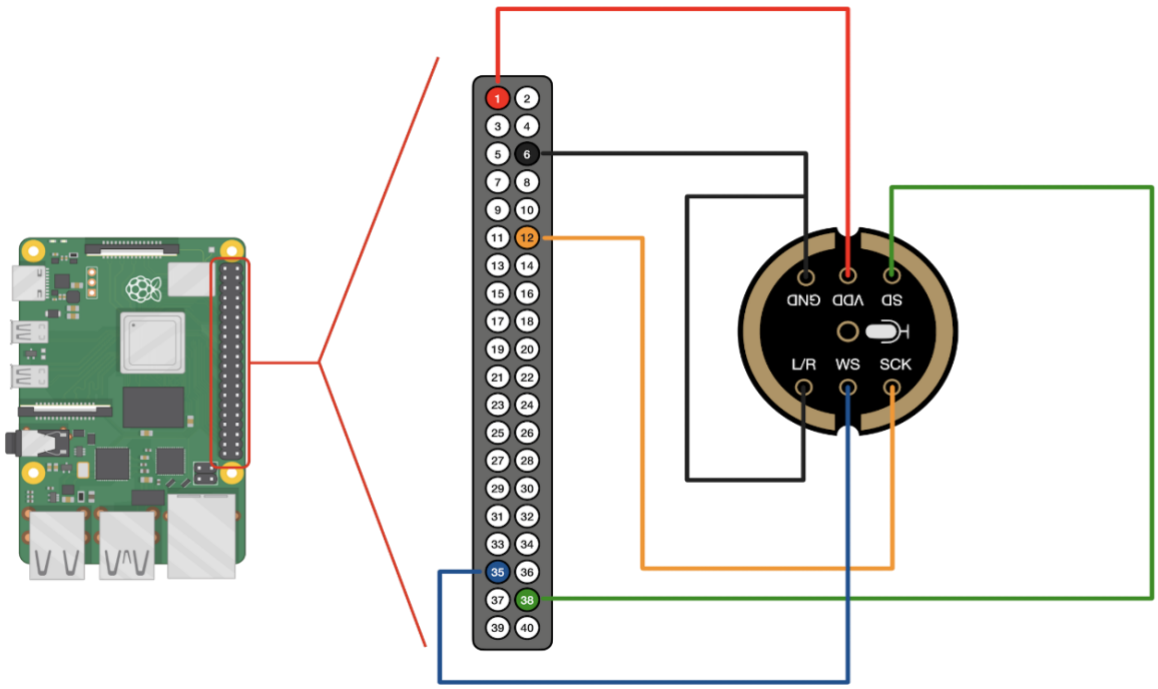

Figure 4: *Hardware Connection Between Raspberry Pi 4B & INMP441*

INMP441 Pin	Raspberry Pi 4B GPIO Pin	Function
VCC	3.3V (Pin 1)	Power Supply
GND	GND (Pin 6)	Ground
SCK (BCLK)	GPIO18 (Pin 12)	Bit Clock
WS (LRCLK)	GPIO19 (Pin 35)	Word Select
SD (DOUT)	GPIO20 (Pin 38)	Serial Data
L/R	GND (Pin 6)	Channel Select

b. Commands Used to Access INMP441 on Raspberry Pi 4B

- `sudo raspi-config`
  → Used to enable the I2S interface in Raspberry Pi.

- `sudo nano /boot/config.txt`
  → Used to add the line *dtparam=i2s=on* to activate I2S in the boot configuration.

- `sudo reboot`
  → Restart the system to apply configuration changes.

- `arecord -l`
  → Used to verify that the INMP441 microphone is detected.

- `arecord -D plughw:1,0 -f S32_LE -r 16000 test.wav`
  → Used to record digital audio from the INMP441 microphone.

3.2. Audio Input & Streaming

The audio module (inmp441 Mic Module) captures real-time microphone input using a streaming-based architecture. Instead of processing audio in large chunks, the system processes smaller buffered frames, enabling:

c. Reduced latency
d. Real-time transcription
e. Continuous listening capability

Streaming-based processing ensures that the assistant responds quickly without noticeable delay.

3.3.Speech Recognition (ASR)

Automatic Speech Recognition was implemented using the Vosk offline speech recognition engine with a Hindi language model.
Key implementation features:

- Streaming recognition
- Partial and final transcript handling
- Lightweight model for reduced memory usage
- Optimized inference for ARM architecture

Unlike cloud-based APIs, Vosk operates entirely offline, making the system privacy-preserving and suitable for low-connectivity environments.

3.4.Intent Detection & NLP

A lightweight rule-based Natural Language Processing (NLP) engine was implemented for primary intent detection, complemented by a cosine similarity–based fallback mechanism.

A two-step intent detection strategy was designed to improve robustness against Automatic Speech Recognition (ASR) variations. Hindi speech transcription often contains minor inconsistencies due to pronunciation differences, regional accents, or orthographic variations (e.g., characters written with or without *nukta*). Additionally, slight discrepancies may arise between the spoken phrase and the ASR-generated text. To address this:

1. **Rule-Based Matching:**
   The system first performs deterministic keyword mapping, normalization, filler-word removal, and regex-based pattern extraction for precise command identification.

2. **Cosine Similarity Fallback:**
   If direct keyword matching fails, cosine similarity is computed between TF-IDF vector representations of the user input and predefined intent templates. This enables approximate semantic matching and improves intent detection accuracy under transcription variations.

This hybrid approach ensures high efficiency while maintaining robustness, without introducing the computational overhead of transformer-based models — making it suitable for edge deployment on ARM-based devices.

Example capabilities include:

- Timer detection *(e.g., "10 सेकेंड का टाइमर लगाओ")*
- Time-related queries
- Basic conversational responses

This approach was chosen over heavy transformer models to ensure low computational overhead.

3.5. State Management

The assistant uses a simple state-machine-based architecture with states such as:

- IDLE
- ACTIVE
- PROCESSING

A timeout mechanism ensures that the assistant exits active mode after a defined period of inactivity. Unknown intent thresholds prevent repetitive misclassification and improve system stability.

State transitions follow a defined control flow: IDLE → ACTIVE → PROCESSING Using the Wake Word, that is common hindi greetings. This design enhances control flow efficiency and makes the system scalable.

3.6. Action Execution Layer

After intent classification, the system triggers the corresponding action module. The action layer is modular, allowing easy addition of new commands without modifying core components.

Examples:

- Timer setup
- Informational responses
- System acknowledgment responses

This modular approach ensures maintainability and extensibility.

3.7. Text-to-Speech (TTS)

The assistant generates speech responses using an offline TTS engine.

Features include:

- No cloud dependency
- Low-latency response generation
- Hindi-compatible output

This ensures complete end-to-end offline functionality.

3.8. Integration & Testing

All modules were integrated into a unified control loop, a common Pipeline Architecuring File was created where the all the Modules are integrated together. Which is then called in the Main file inside a loop.

Testing involved:

- Functional validation of commands
- Latency measurement

- CPU and memory monitoring
- Edge-case handling for incorrect inputs

The system was iteratively refined to improve responsiveness and stability.

4. Optimization & Performance

4.1. Edge Deployment on ARM SoC

The system was specifically designed and optimized for deployment on an ARM-based single-board computer. Unlike traditional cloud-dependent AI systems, this assistant operates entirely offline, leveraging on-device computation.
To ensure compatibility and performance:

- A lightweight Hindi ASR model was selected.
- All processing was executed locally without cloud API calls.
- Modular architecture minimized unnecessary computation.
- Dependencies were kept minimal to reduce system overhead.

This design ensures that the assistant can function reliably in environments with limited connectivity while preserving user privacy.

4.2. Memory Optimization

Given the limited RAM available on edge devices, memory efficiency was a key design consideration.
Optimization steps included:

- Using the small Vosk Hindi model instead of larger alternatives.
- Avoiding large transformer-based NLP models.
- Releasing unused buffers after processing.
- Keeping the NLP engine rule-based and lightweight.
- Minimizing background threads and redundant processes.

As a result, the system maintained stable memory usage during continuous operation.

4.3. CPU Optimization

Real-time responsiveness was critical for user experience. CPU usage was optimized through:

- Streaming audio processing instead of batch processing.
- Event-driven command execution.
- Limiting continuous polling loops.
- Efficient state management logic.
- Avoiding unnecessary re-initialization of models.

The ASR model is loaded once during startup and reused for all inference operations, significantly reducing overhead.

4.4. Latency & Resource Usage

Performance evaluation was conducted under real-time usage conditions.
Key observations:

- Average response latency: ~300–700 ms after speech completion
- CPU usage during inference: moderate utilization on ARM cores
- Stable operation during extended testing
- No dependency on internet connectivity

The streaming-based pipeline ensures that transcription begins while audio is still being captured, reducing perceived delay.

A Sample benchmarks is given below:

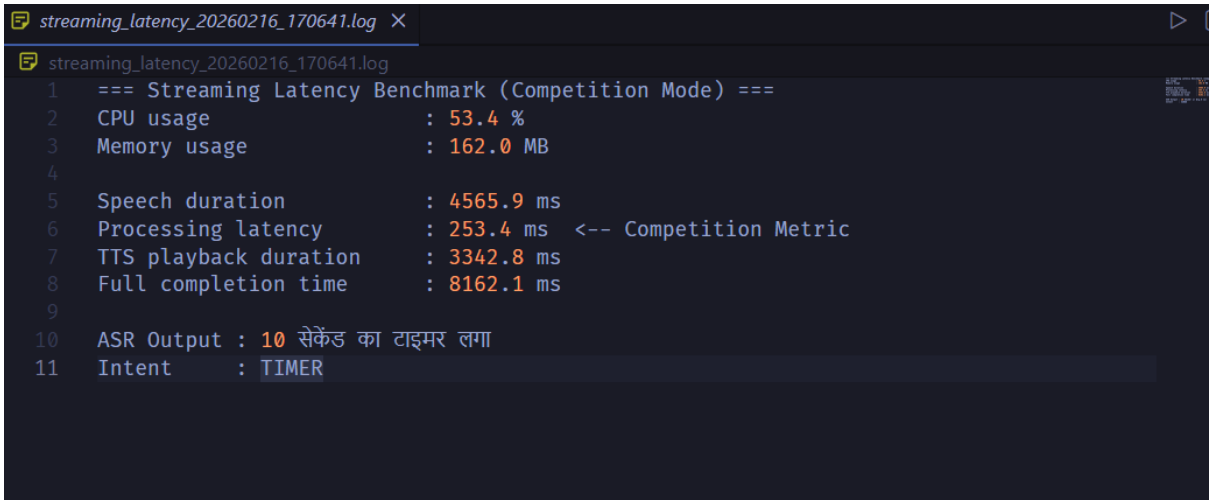

Figure 5: *Sample Benchmark, Intent: TIMER*

4.5. Reliability & Stability Improvements

To enhance robustness:

- A timeout mechanism prevents indefinite active listening.
- Unknown intent threshold prevents repetitive misclassification.
- Input normalization reduces parsing errors.
- Modular error handling ensures system continuity.

These mechanisms improve long-term usability and prevent system crashes.

5. Results & Challenges

5.1. System Performance

The offline Hindi Voice Assistant was successfully deployed and tested on an ARM-based single-board computer. The system demonstrated stable real-time performance under continuous usage.

Key outcomes:

- Successful real-time Hindi speech recognition
- Accurate detection of predefined intents
- Fully offline execution without internet dependency
- Stable operation during extended runtime testing
- Modular and extensible system design

The assistant responded to user commands with minimal delay, providing a smooth and interactive experience.

5.2. Performance Observations

During evaluation, the following observations were recorded:

- Average response latency: approximately 300–700 ms after speech completion
- Consistent CPU utilization during ASR inference
- Stable memory usage without memory leaks
- No crashes observed during prolonged testing

The streaming-based ASR significantly reduced perceived delay by processing audio in real time rather than waiting for complete speech input.

5.3. Accuracy Observations

The system achieved reliable performance for:

- Timer-based commands
- Time-related queries
- Basic conversational responses

However, ASR accuracy varied depending on:

- Pronunciation clarity
- Background noise
- Speaking speed

Despite these variations, the rule-based NLP engine successfully extracted key intent keywords in most cases.

5.4. Challenges Faced

Several technical challenges were encountered during development:

1. Hindi ASR Accuracy
Handling variations in pronunciation and regional accents required careful preprocessing and normalization.

2. Intent Ambiguity
Similar phrases with slight wording differences required flexible keyword handling and pattern matching.

3. Resource Constraints
Running ASR models on ARM hardware required selecting lightweight models and optimizing memory usage.

4. Latency Optimization
Initial versions experienced delay due to buffering strategies, which were later improved using streaming recognition.

5. Integration Complexity
Ensuring smooth communication between audio streaming, ASR, NLP, and TTS required careful state management and debugging.

5.5. Limitations

While functional, the system currently has certain limitations:

- Limited intent coverage (rule-based system)
- No deep contextual conversation handling
- ASR accuracy dependent on environment
- No speaker recognition capability

These limitations provide clear directions for future improvement.

6. Conclusion & Future Scope

6.1. Conclusion

This project successfully demonstrates the design and deployment of a fully offline Hindi Voice Assistant optimized for ARM-based edge devices. By integrating lightweight speech recognition, rule-based natural language processing, modular system architecture, and offline text-to-speech synthesis, the system achieves real-time voice interaction without reliance on cloud infrastructure.

The assistant operates entirely on-device, ensuring enhanced privacy, reduced latency, and reliable performance in low-connectivity environments. The modular implementation approach improves maintainability and scalability, making it adaptable for future enhancements.

Through careful optimization of CPU and memory usage, the project highlights the practical feasibility of deploying AI applications on resource-constrained SoC platforms. The implementation reinforces the importance of Edge AI in enabling accessible, secure, and efficient intelligent systems.

Overall, the project demonstrates how AI-driven solutions can be effectively engineered for real-world deployment using ARM-based hardware.

6.2. Future Scope

While the current system achieves its intended objectives, several enhancements can further improve functionality and performance:

- Integration of lightweight machine learning–based intent classification
- Wake-word detection using optimized edge models
- Multi-language support beyond Hindi
- Speaker recognition for personalized responses
- Smart home device integration
- Noise-robust ASR improvements
- Deployment using model quantization techniques for further optimization

Future development can transform the assistant into a more intelligent, adaptive, and scalable edge AI system.

7. References

1. Vosk Models: *https://alphacephei.com/vosk/models*

2. eSpeakNg: *https://github.com/espeak-ng/espeak-ng?tab=readme-ov-file#documentation*

3. Python 3.10.12: https://www.python.org/

4. Raspberry Pi Foundation. *Raspberry Pi 4 Model B Technical Specifications.* Available at: *https://www.raspberrypi.com/products/raspberry-pi-4-model-b/*

5. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval.* Cambridge University Press. (Reference for TF-IDF and Cosine Similarity)

6. ARM Ltd. *ARM Architecture Reference Manual.* Available at: *https://developer.arm.com*

7. Robo.in: https://robu.in/product/inmp441-mems-high-precision-omnidirectional-microphone-module-i2s/

Appendix

A. Project Directory Structure:

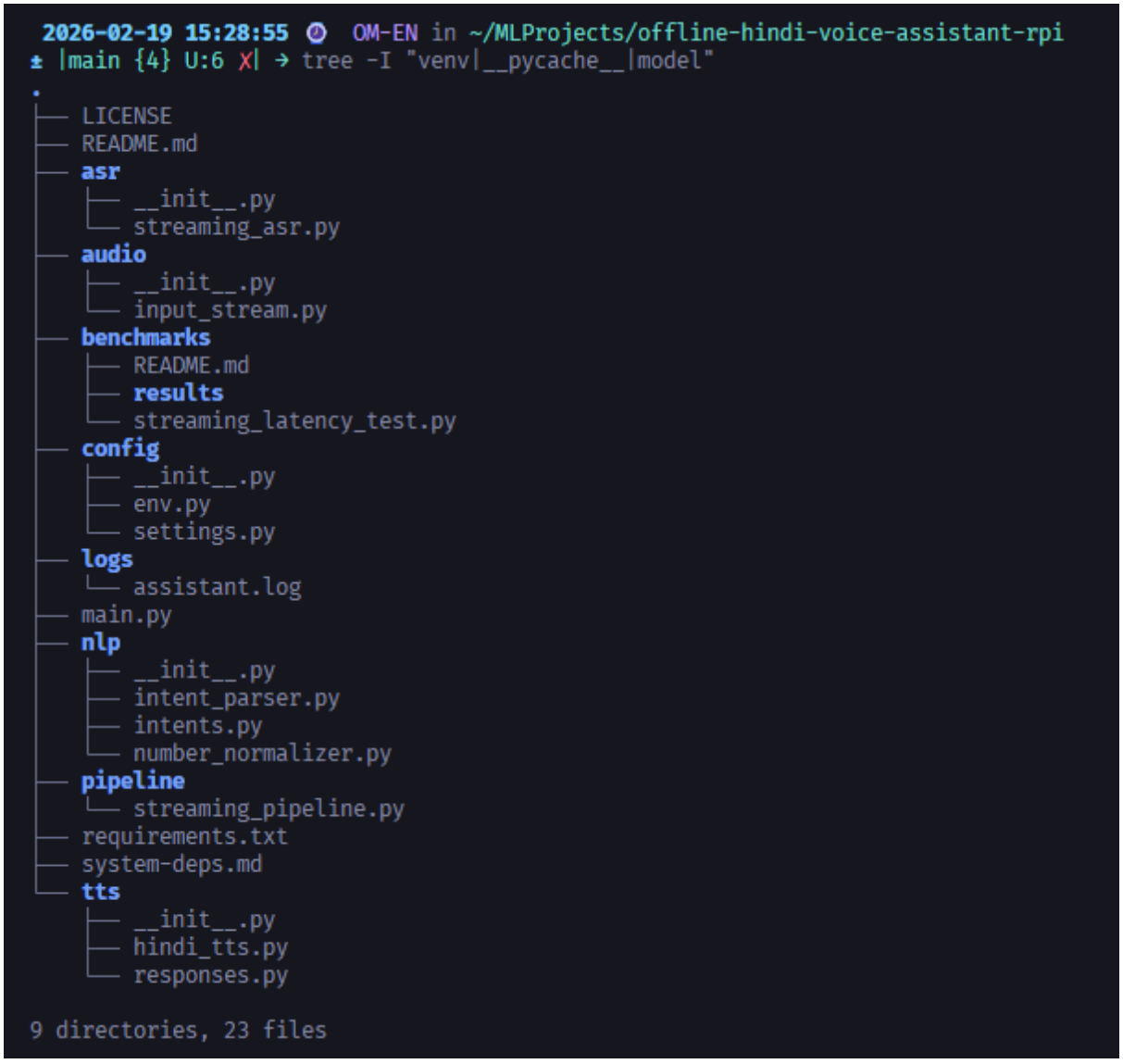

Figure 6: *Project Directory Structure*

B. GitHub Repo Link

https://github.com/PushkarOM/offline-hindi-voice-assistant-rpi

C. Demo Vide Link

https://youtu.be/ryDRLKcCcr8?si=5ujjtuz32ikEVcwN

D. Sample Benchmarks

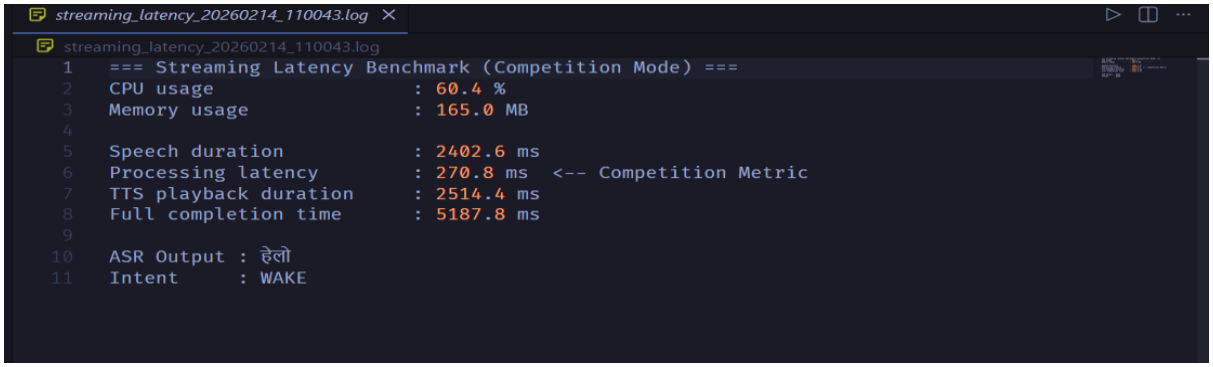

Figure 7: *Sample Benchmark, Intent: WAKE*

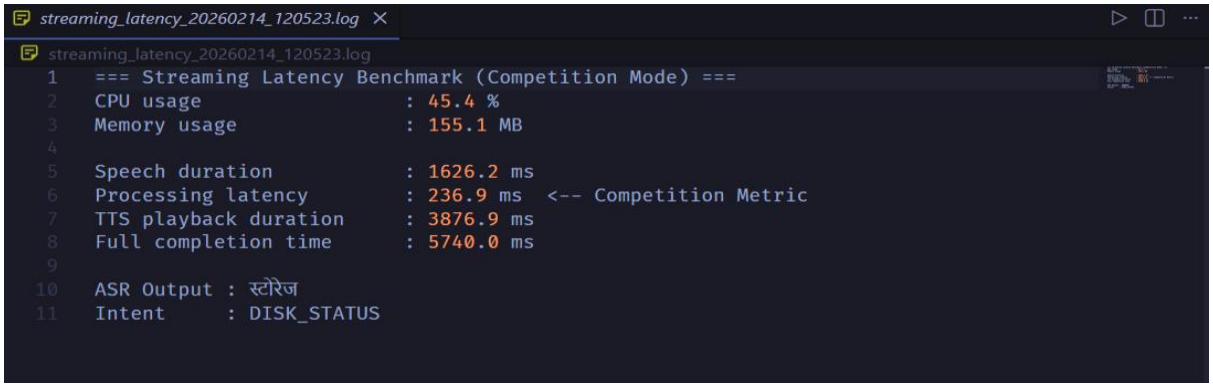

Figure 8: *Sample Benchmark, Intent: DISK_STATUS*

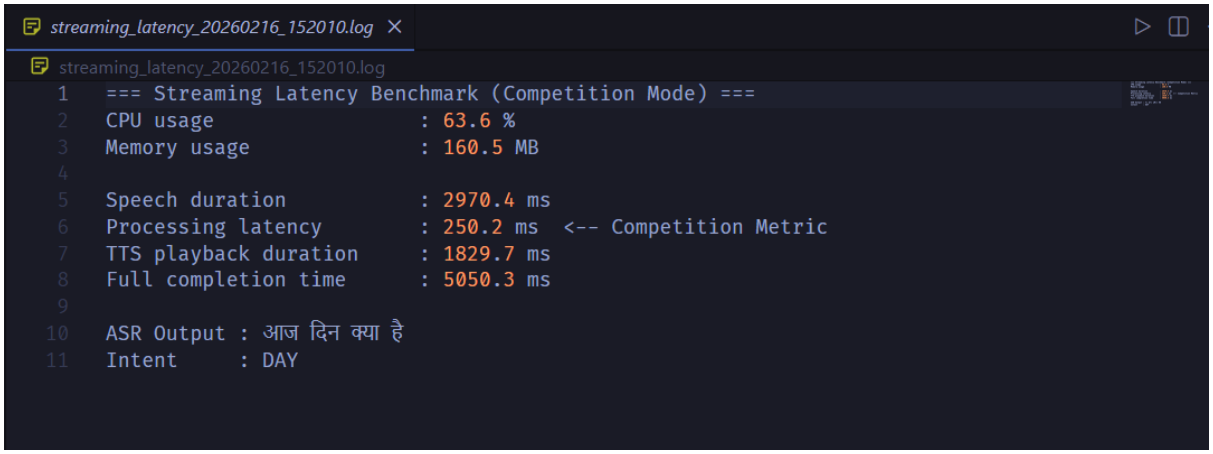

Figure 9: *Sample Benchmark, Intent: DAY*

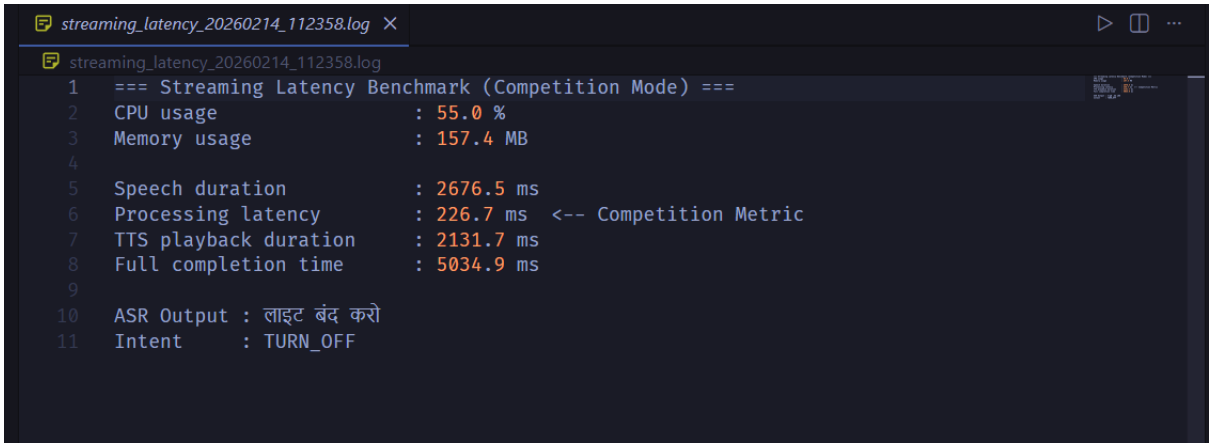

Figure 10: *Sample Benchmark, Intent: TURN_OFF*